Disk-Resident Graph Indexes for Billion-Scale Approximate Nearest Neighbor Search

Zalán Varga
University of Auckland
Auckland, New Zealand
zvar509@aucklanduni.ac.nz

Jadon Chu
University of Auckland
Auckland, New Zealand
jchu630@aucklanduni.ac.nz

Xiangrong Jiang
University of Auckland
Auckland, New Zealand
xjia340@aucklanduni.ac.nz

Igor Kim
University of Auckland
Auckland, New Zealand
ikim810@aucklanduni.ac.nz

***Abstract*—Approximate nearest neighbor search (ANNS) over high-dimensional vectors is the retrieval step behind web search, recommendation and retrieval-augmented generation. Graph indexes such as HNSW and NSG give the best recall per distance computation, but they keep the graph and the raw vectors in DRAM, so a billion points needs several machines or compression that costs recall. DiskANN, our seed paper, builds the graph for disk instead: a pruning parameter $\alpha > 1$ yields a graph that needs fewer hops, product-quantized vectors stay in memory, and full-precision vectors sit beside their adjacency lists on an SSD, so one 64 GB machine serves a billion points. We review seven papers around that design: the in-memory graphs it set out to beat (HNSW, NSG), DiskANN itself, the inverted-index alternative SPANN, and three later responses that change the disk layout (Starling), the query semantics (Filtered-DiskANN) and the hardware (BANG). We compare what each keeps in memory, what each read buys, build cost and evaluation setup, and show where reported speedups cannot be compared. We then test whether pruning and disk layout should be designed together. In a simulation on one million synthetic 128-dimensional points, Starling-style layout with block search cut block reads at recall@10 = 0.95 by 3.0× relative to DiskANN's layout and vertex search, but the better of our two layout-aware pruning rules saved only 2–7% of reads beyond an ordinary extra construction pass at 10^5 points and nothing measurable at 10^6.**

***Index Terms*—Approximate nearest neighbor search, Graph-based index, Disk-resident index, Product quantization, Vector database, Billion-scale retrieval**

I. Introduction

Nearest neighbor search over learned embeddings is now a standard component of information systems. Web search and advertising retrieve documents and ads whose embeddings are close to a query embedding [1], [2], recommender systems recall candidate items the same way, and retrieval-augmented language models fetch supporting passages by vector similarity [3]. In every case the entities are mapped into a space of a hundred to a few hundred dimensions, and "similar" means "close".

Two facts make this hard at scale. First, exact search in high dimension essentially requires a linear scan, the curse of dimensionality [4], [5], so practical systems return *approximate* neighbors and are judged by how much recall they trade for speed. Second, the data are large: one billion 128-dimensional SIFT descriptors occupy 128 GB as bytes, and one billion 96-dimensional DEEP vectors occupy 384 GB as floats [6], before any index is built. Before 2019 there were two ways to index a billion points [5]. Inverted indexes with product quantization fit in under 64 GB but plateau at roughly 0.5 1-recall@1 because the compression is lossy. Alternatively, the data can be split across machines that each hold an in-memory graph; an NSG index over 100 million 128-dimensional floats needs about 75 GB, and Taobao's deployment split two billion points over 32 machines [5], [7]. Simply moving a graph index to an SSD does not work either: a naive mapping costs several hundred random reads per query, and a retail SSD takes a few hundred microseconds per random read [5].

DiskANN [5] showed that a graph index *can* live on an SSD if the graph is built for it. This report reviews DiskANN, the work it responded to, a rival design from the same period, and three later papers that respond to it. We organize the review around one question: *what does each system keep in memory, and what does each step of its search cost?* Our objectives are to explain how each method works (with worked examples), to compare the methods on a common cost model rather than on their headline numbers, to state plainly where the reported evidence is and is not comparable, and to propose and test an improvement. Our initial hypothesis, submitted with our Stage 1 abstract, was that the post-DiskANN literature optimizes four layers separately (the pruning rule, the block layout, the query semantics and the hardware) and that designing the first two together would reduce disk reads. We test that hypothesis in Section V, and the result is more qualified than we expected.

Section II sets up the problem and the tools every paper uses. Section III reviews the seven papers. Section IV compares them and their evidence. Section V presents our proposals and a small experiment, Section VI discusses open problems, and Section VII concludes.

II. Background and Problem Setting

A. Problem and metrics

Given a dataset P of n points in $\mathbb{R}^d$ and a query q, the k -nearest-neighbor problem asks for the k points of P closest to q. If an algorithm returns a set X of k points and G is the true set, its k-recall@k is $|X \cap G| \,/k$ [5]. Every paper we review reports a curve of recall against queries per second (QPS) or latency, obtained by sweeping a search parameter. The curves are not interchangeable: DiskANN mostly reports 1-recall@1, while SPANN, Starling, Filtered-DiskANN and BANG report 10-recall@10 or similar, and a system can be strong on one and weak on the other.

B. Index families

There are four broad families. Locality-sensitive hashing, covered in this course, gives worst-case guarantees on the trade-off between index size and query time [4], but it does not adapt to the structure of real data and is outperformed by graph methods on real benchmarks [5]. Space-partitioning trees are compact and fast below about 20 dimensions and slow above it [5]. Quantization methods with inverted files compress vectors so that billions fit in memory, at a cost in recall. Proximity graphs connect each point to a few nearby points and answer a query by walking the graph toward it; they lead in-memory benchmarks [7], [8], and all of the graph systems we review share the walk and one pruning rule, described next.

C. Greedy search and the pruning rule

GreedySearch keeps a candidate list of size L, initially a fixed start node. It repeatedly takes the closest candidate not yet expanded, adds that vertex's neighbors to the list, and trims the list back to the L closest, stopping when every candidate has been expanded [5]. A larger L raises recall and cost.

Which edges a graph keeps decides how quickly this walk converges. Pruning rules descended from the relative neighborhood graph [9] keep an edge from p to a candidate only if no already-kept neighbor of p is closer to that candidate than p is. DiskANN generalizes the rule with a parameter $\alpha \geq 1$ [5]. Its *RobustPrune*(p, V, α, R) sorts the candidate set V by distance to p, repeatedly keeps the closest remaining candidate p^*, and removes every candidate p' that p^* "covers":

$$\alpha \cdot d(p^*, p') \leq d(p, p'), \tag{1}$$

stopping when R neighbors have been kept. With $\alpha = 1$ an edge is dropped whenever a detour through a kept neighbor makes *any* progress toward the candidate. With $\alpha > 1$ the detour must shrink the remaining distance by a factor of at least α, otherwise the direct, longer edge is kept. DiskANN observes that HNSW and NSG both implicitly use $\alpha = 1$ [5].

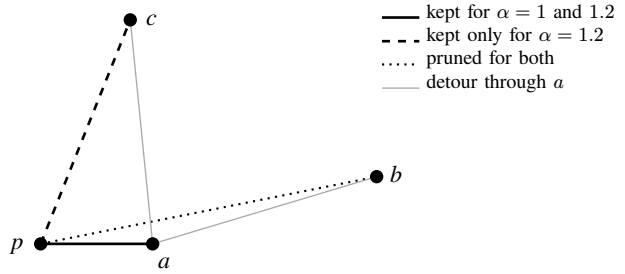


Fig. 1. Example 1: RobustPrune at $p = (0, 0)$ with candidates $a = (1, 0)$, $c = (0.8, 2)$ and $b = (3, 0.6)$.

Example 1 (running example). In Fig. 1, the candidates of p sorted by distance are a ($d = 1.000$), c (2.154) and b (3.059). RobustPrune keeps a first and then tests the others against (1), using $d(a, c) = 2.010$ and $d(a, b) = 2.088$. With $\alpha = 1$, both tests succeed ($2.010 \leq 2.154$ and $2.088 \leq 3.059$), so c and b are dropped and p keeps only a. A search from p toward c must then go through a, and that hop only reduces the remaining distance from 2.154 to 2.010, about 7%. With $\alpha = 1.2$ the test for c fails ($1.2 \times 2.010 = 2.412 > 2.154$), so the direct edge to c is kept, while b is still dropped ($1.2 \times 2.088 = 2.506 \leq 3.059$) because going through a cuts its distance by a third. With $\alpha = 2$, b is kept as well ($2 \times 2.088 = 4.176 > 3.059$); DiskANN's own builds use $\alpha = 1.2$ [5]. In memory a wasted hop costs a few cache misses; on an SSD it costs a round trip. We reuse this example in Section III.

D. Product quantization

Product quantization (PQ) [10] splits each d-dimensional vector into m sub-vectors, runs k-means with 256 centroids in each subspace, and stores each sub-vector as the one-byte index of its nearest centroid, so a vector costs m bytes. A query's distance to every centroid is computed once into a lookup table, after which the approximate distance to any stored vector is a sum of m table entries. DiskANN's example uses 32-byte codes, 32 GB for a billion points [5]; Starling and BANG use the same idea [3], [6].

E. Storage facts

An SSD reads in blocks, typically 4 KB, so reading 512 bytes costs the same as reading 4 KB [5]. A retail SSD serves a random read in a few hundred microseconds and about 300,000 random reads per second, and a small batch of random reads takes about as long as one [5]. Two consequences follow: latency is governed by the number of *dependent* round trips, and throughput by the total number of reads. The size of one vertex record decides how many fit in a block. With byte-valued 128-dimensional vectors and at most 31 neighbor IDs, a record is $128 + 4 + 31 \times 4 = 256$ bytes and a block holds 16 records, the configuration Starling uses on BIGANN [3]; with DiskANN's degree bound of 128 the neighbor list alone is 512 bytes [5].

III. Literature Review

A. Before the seed: HNSW and NSG

HNSW [8] builds a hierarchy of proximity graphs. Each inserted element draws a top layer $l = \lfloor -\ln(U) \cdot m_L \rfloor$ with U uniform on $(0, 1)$, so layer membership decays exponentially; the authors recommend $m_L = 1/\ln(M)$, the analogue of a skip list with $p = 1/M$. A search starts at the top layer, descends greedily with a candidate list of one, and runs a full search with list size ef at layer 0. Neighbors are chosen by a heuristic that adds candidate e only if e is closer to the new element than to every neighbor already chosen, which is (1) with $\alpha = 1$. Elements are inserted one at a time, so the index grows incrementally. The authors report logarithmic scaling of search cost on low-dimensional data and a strong lead on public benchmarks. Its costs are memory and, for our purposes, the absence of any disk story: vectors and all layers' links live in RAM. On a 200M-point SIFT subset HNSW used 64 GB at peak and built in 42 minutes to 5.6 hours depending on parameters, against 23.5–30 GB for FAISS [8], and Starling estimates that HNSW over a billion 96-dimensional floats would need more than 350 GB [3].

NSG [7] drops the hierarchy. Its authors define the monotonic relative neighborhood graph (MRNG), which guarantees a monotonic search path between any two nodes and close-to-logarithmic expected search time, and approximate it cheaply. NSG first builds an approximate k-NN graph, picks the point nearest the centroid as a single *navigating node* from which every search starts, and for each point runs a search from that node, collects the visited vertices as candidates and selects at most m neighbors with the MRNG rule. A depth-first spanning tree from the navigating node then reconnects any vertex the degree bound cut off. The result is a single sparse graph with no auxiliary structure and, in the authors' experiments, the smallest memory footprint among graph methods. NSG is integrated into Taobao's billion-scale search engine [7]. The costs are visible in its largest experiment: on 100 million DEEP vectors the k-NN graph took 6.75 hours on four GPUs and NSG itself another 9.7 hours, with 92 GB peak memory during indexing and 55 GB during search, while HNSW ran out of memory on the same 96 GB machine [7].

Assessment. Both papers optimize recall per distance computation in RAM, and both are very good at it; Vamana's own in-memory comparison finds NSG and Vamana ahead of HNSW [5]. Neither is built for a setting in which each hop is an I/O. In terms of Example 1, both keep only the edge to a, and in RAM the extra hop through a is nearly free. *Strengths:* the best recall per distance computation in RAM, and HNSW accepts inserts. *Limitations:* every vector and link lives in DRAM, NSG's build depends on an expensive k-NN graph, and neither has a disk design.

B. The seed: DiskANN

Construction. Vamana [5] starts from a random graph in which every vertex has R out-neighbors, and uses the dataset medoid s as the start node. It visits the points in random order. For each point p it runs GreedySearch$\big(s, x_p, 1, L\big)$ on the current graph, takes the set V of all vertices the search visited, sets p's out-neighbors to RobustPrune(p, V, α, R), and adds the reverse edges, re-pruning any neighbor whose degree now exceeds R. It makes two passes, the first with $\alpha = 1$ and the second with the user's $\alpha > 1$. Three choices separate it from its predecessors: the α parameter; the use of the whole visited set as candidates (as in NSG, unlike HNSW's L results), which supplies far-away candidates; and a random rather than k-NN or empty initial graph. The payoff is measured in hops: to reach 98% 5-recall@5 on SIFT1M, Vamana needs two to three times fewer hops than HNSW and NSG, and its hop count keeps falling as R and α grow while theirs stagnates [5]. Used in memory, Vamana matches or beats both, and on DEEP1M it built in 129 s against 219 s for HNSW and 480 s for NSG.

Scaling the build. A billion-point Vamana graph built in one shot needed about 1,100 GB of RAM and two days [5]. DiskANN instead partitions the data into $k = 40$ clusters with k-means, assigns each point to its $\ell = 2$ nearest clusters, builds an in-memory Vamana graph for each overlapping shard (about $n\ell/k$ points, here 50 million), and takes the union of the edge sets. Because shards overlap, the merged graph stays navigable across shard boundaries. For SIFT1B the merged index is 348 GB with an average degree of 92.1 and took about five days on a workstation with 16 cores and 64 GB of RAM; it needs at most about 20% more latency than the one-shot index for the same recall.

Serving. PQ codes for every point stay in memory. On the SSD, each vertex's full-precision vector is stored next to its neighbor list, padded to a fixed size so that a vertex's offset can be computed from its ID. Search is *BeamSearch*: instead of expanding one vertex per round trip it fetches the neighbor lists of the W closest unexpanded candidates in one batch ($W = 2$–8 works well), guiding the walk with PQ distances. Because a vertex's full vector arrives in the same block as its neighbors, the search re-ranks its final candidates with exact distances at no extra I/O. Vertices within three or four hops of s are cached in RAM.

Results and costs. On SIFT1B, DiskANN serves more than 5,000 queries per second at under 3 ms mean latency with 95%+ 1-recall@1 on a 16-core machine [5]. At the same memory footprint, IVFOADC+G+P plateaus at 62.74% 1-recall@1 with 32-byte codes, while DiskANN reaches 95% in under 3.5 ms. The single-shot index reaches 98.68% 1-recall@1 in under 5 ms. The costs are equally concrete. The build is long, offline and static: the paper has no insert or delete path. Each beam read fetches a full block for one vertex. Routing uses PQ distances, so a search can be led astray before re-ranking. And the SSD must be run at a 30–40% load factor to keep latency low, with each search thread spending 40–50% of its time in I/O. *Strengths:* a billion points at 95%+ recall on one 64 GB machine, with few hops per query. *Limitations:* a multi-day static build, one useful vertex per block read, routing errors from PQ, and an SSD that must stay lightly loaded.

C. A rival paradigm: SPANN

SPANN [1] answers the same question, a billion points on one node with little memory, without a graph over the data. Memory holds only the centroids of N *posting lists*, each represented by the real vector nearest its centroid and indexed by an in-memory SPTAG index; the posting lists themselves sit on disk. A query finds its nearest centroids and reads their posting lists in one round. Making this accurate requires three fixes. *Hierarchical balanced clustering* bounds the length of every posting list (12 KB for byte vectors, 48 KB for floats) so each can be read in a few blocks, while reducing clustering cost from $O(|X|\ dN)$ to $O(|X|\ dk \log_k N)$ for a small branching factor k. *Closure assignment* duplicates boundary vectors: x joins list j if $d\big(x, c_j\big) \le (1+\varepsilon_1)d(x, c_1)$, with at most 8 replicas, and an RNG-style rule skips a list whose centroid lies behind an already chosen one, so replicas go to lists in different directions. *Query-aware pruning* reads list j only if $d\big(q, c_j\big) \le (1+\varepsilon_2)d(q, c_1)$, so easy queries read fewer lists.

At matched memory (about 32 GB for SIFT1B and SPACEV1B, with centroids for 10–12% of the points), SPANN reports more than twice DiskANN's speed at 90% recall on three billion-point datasets, and 90% recall@1 and recall@10 in about 1 ms [1]. It is deployed in Bing. Its advantages are different in kind from DiskANN's: the reads of one query are independent rather than a chain, and the design extends naturally to distributed search, where it cut the machines contacted per query from 32 to 6.3. Its main cost is disk space, since each vector may be stored up to eight times. In Starling's evaluation, with disk capped at 10 GB, SPANN needed 50 ms to reach 0.95 recall where DiskANN needed 10 ms, and SPANN improved as the cap was raised to 32 GB [3]. That comparison is fair to its setting but not neutral, since the cap is exactly what disables SPANN's main lever. SPANN reports building a SIFT1M index in about two minutes but gives no billion-scale build time, so "cheaper to build than DiskANN", which our initial hypothesis assumed, is not something the evidence settles. *Strengths:* one round of independent reads, memory of about a tenth of the raw data, and a natural path to distributed search. *Limitations:* up to 8× duplication on disk, weak performance when disk is capped, and no reported billion-scale build time.

D. After the seed: layout, semantics and hardware

Starling [3] targets the *segments* into which vector databases split data, each with under 2 GB of memory and 10 GB of disk (33 million BIGANN vectors per segment). It measures that disk I/O takes up to 92.5% of DiskANN's search time there, and that DiskANN's blocks are laid out by vertex ID, so the 15 other records in each 4 KB block are unrelated to the query. Its remedies work on three fronts. *Block shuffling* reorders vertices so that neighbors share blocks. Starling measures locality with the overlap ratio $\mathrm{OR}(G)$, the average fraction of a vertex's block-mates that are also its out-neighbors, proves that maximizing it is NP-hard and, unless P = NP, has no polynomial-time approximation with a finite factor, and gives three heuristics, recommending Block Neighbor Frequency (BNF), which repeatedly moves each vertex to the block holding most of its neighbors; shuffling costs 3–12% of graph construction time. An *in-memory navigation graph* on a sample of under 10% of the points supplies query-specific entry points, cutting I/Os by about 20%. *Block search* uses everything a read returns: it computes distances to all vertices in the loaded block, expands the closest fraction $\sigma = 0.3$ of them, and pipelines the next read with this computation. On BIGANN this lifts the vertex utilization ratio ξ (useful vertices per loaded block) from 0.0625 to 0.344 and cuts the search path from 362 to 182 vertices. For standard top-10 search, Starling reaches 0.95 recall in 5 ms against DiskANN's 10 ms, about twice DiskANN's throughput at 0.99 recall (538 vs. 239 QPS), and more than twice at billion scale above 0.96 recall. The headline 43.9× is for *range* search at average precision 0.9, where DiskANN must reissue searches [3]. *Strengths:* about half DiskANN's reads and latency with no extra disk, and a layout that works on any graph. *Limitations:* the layout problem is NP-hard and solved heuristically, it must be redone after every rebuild, and $\mathrm{OR}(G)$ falls as segments grow.

We add one observation. The ξ that Starling reports for DiskANN equals $1/\varepsilon$ on all four datasets (0.0625 = 1/16, 0.1429 = 1/7, 0.1111 = 1/9, 0.25 = 1/4, where ε is the number of records per block), and Starling's 0.344 on BIGANN equals $(1 + 0.3 \times 15)/16$, the target vertex plus the 30% of block-mates that block search expands; on the other three datasets Starling's ξ is higher than this rule gives. So the "94% of each block wasted" figure follows from record size and the search rule rather than being an empirical discovery; the wasted bandwidth is real, but the informative measurements are path length, reads and latency.

Filtered-DiskANN [2] changes the query instead. Each point carries a set of labels, and a query asks for its nearest neighbors among points carrying a given label f; the *specificity* of f is the fraction of points that carry it. Post-filtering an ordinary search fails when specificity is low, because almost all returned neighbors are rejected. *FilteredGreedySearch* starts from per-label start points and only traverses vertices that share a label with the query. *FilteredVamana* builds the graph incrementally with a filter-aware RobustPrune, in which an edge (p, p') may be pruned through p^* only if (1) holds *and* $F_p \cap F_{p'} \subseteq F_{p^*}$, so that no label loses the paths it needs. In Example 1, if p and c share a label that a lacks, the edge from p to c survives even at $\alpha = 1$: a search filtered on that label cannot pass through a, so a cannot cover c. *StitchedVamana* instead builds a small Vamana graph per label, overlays them and prunes the union with the same rule. At 90% recall, FilteredVamana and StitchedVamana deliver 2.5× and 6× the QPS of the best prior method (inline-filtered IVF) on one ad dataset and 3× and 7.5× on another, and both keep high recall at specificities down to 10^{-6} where baselines fail. StitchedVamana is about twice as fast to search but slower to build and cannot take inserts. On unfiltered queries the filtered indexes reach 95% recall at 0.8–0.9× Vamana's QPS. Deployed in sponsored search across 47 regions, FilteredVamana raised clicks by 34.6% and revenue by 49.0% [2]. Most experiments use 1–3 million points in memory, with one SSD-resident run on 28 million points. Each query carries one label; disjunc-

TABLE I
DESIGN COMPARISON: WHAT EACH SYSTEM KEEPS WHERE, AND WHAT A QUERY DOES

System	Structure and build	In fast memory	On SSD	Per-query access	Updates
HNSW	Layered graphs, heuristic pruning ($\alpha = 1$); one incremental pass	Vectors + all layers	—	Random RAM accesses, ~log hops	Inserts
NSG	MRNG approximation over a k-NN graph; DFS repair	Vectors + graph	—	Random RAM accesses	Rebuild
DiskANN	Vamana, $\alpha > 1$, 2 passes; merged overlapping shards	PQ codes (32 B/pt) + cache	Vectors + adjacency	Chain of beam reads (W = 2–8)	Rebuild
SPANN	Balanced clusters, $\leq$8 replicas of boundary points	Centroids (10–12% of pts)	Posting lists	One round of list reads	Rebuild
Starling	Any graph + BNF shuffling (+3–12% build)	PQ + <10% nav. graph + block map	Reordered graph	Fewer reads, block search	Rebuild + reshuffle
Filtered-DiskANN	Vamana with label-aware pruning	As DiskANN (or all in RAM)	As DiskANN	Walk restricted to label	Inserts (Filtered-Vamana)
BANG	None (reuses Vamana)	GPU: PQ (74 B/pt); host: graph + vectors	—	PCIe transfer per iteration	—

TABLE II
WHAT EACH PAPER MEASURED: THE SETUP BEHIND EACH HEADLINE

Paper	Data and scale	Hardware	Compared against	Headline as reported
HNSW	Million-scale benchmarks; 200M SIFT	CPU server, 128 GB	NSW, FLANN, Annoy, FAISS	Leads in-memory benchmarks
NSG	Million-scale sets; DEEP100M	CPU, 96 GB (+4 GPUs for k-NN)	HNSW, FAISS, others	Best in-memory trade-off; Taobao
DiskANN	SIFT1B, DEEP1B (1-recall@1)	16 cores, 64 GB, two 1 TB consumer SSDs	IVFOADC+G+P; HNSW, NSG in memory	>5k QPS, <3 ms, 95%+ 1-recall@1
SPANN	SIFT1B, DEEP1B, SPACEV1B	52 cores, 128 GB, RAID-0 SSD	DiskANN (rerun by SPANN)	2× DiskANN at 90% recall, matched memory
Starling	33M-vector segments; 1B in 31 segments	8 vCPU NVMe instance	DiskANN, SPANN (in segment caps)	~2× DiskANN on top-k; 43.9× on range search
Filtered-DiskANN	1–3M labeled points; 28M on SSD	64 vCPU, 500 GB	IVF, HNSW, NHQ, Milvus	2.5–7.5× QPS at 90% recall; A/B test
BANG	SIFT1B, DEEP1B, MSSPACEV1B, T2I-1B	A100 80 GB + 660 GiB host	GGNN, FAISS on 1 GPU	50–400× at recall 0.9 (not vs. CPU DiskANN)

tions can be answered by merging per-label searches, and conjunctions are left to future work [2]. *Strengths:* native label filtering down to specificity 10^{-6}, validated in production. *Limitations:* one label per query, a 10–20% QPS cost on unfiltered queries, and mostly in-memory evaluation.

BANG [6] moves search onto a single GPU. A billion-point Vamana graph is 260 GB and SIFT1B's vectors another 128 GB, against 80 GB on an A100. BANG therefore keeps the graph and the full vectors in *host* memory and only PQ codes (74 GB for a billion points) on the GPU. In each iteration the CPU looks up the adjacency list of every query's current candidate and sends it over PCIe; the GPU drops already-visited vertices using a per-query Bloom filter, computes PQ distances, sorts and merges them into the candidate list, and picks the next candidate, while full vectors for final re-ranking are copied asynchronously. Batches of thousands of queries keep the GPU busy. BANG does not build an index; it searches a Vamana graph built by DiskANN. With a single A100 it reports 55×, 50× and 400× the throughput of GGNN and FAISS at 10-recall@10 = 0.9 on SIFT1B, DEEP1B and MSSPACEV1B. Those baselines are GPU methods running on one GPU; BANG is not compared with CPU DiskANN, and on 100M-point datasets that fit in GPU memory, CAGRA achieves 1.5–2× BANG's throughput. Its host has 660 GiB of RAM, because BANG brings the graph back into DRAM: it is a memory-resident index with GPU arithmetic, not a disk index. *Strengths:* very high batch throughput on a single GPU. *Limitations:* 660 GiB of host memory, no comparison with CPU DiskANN, and slower than CAGRA when the data fit in GPU memory.

IV. COMPARATIVE DISCUSSION

Table I summarizes the designs and Table II the evidence behind each paper's headline. Four comparisons stand out.

Memory against reads. HNSW and NSG keep the full vectors and graph in RAM, several hundred bytes per point. DiskANN reduces memory to a 32-byte PQ code per point plus a small cache and pays with a chain of dependent reads; SPANN at matched memory spends its 32 bytes per point on centroids and pays with duplicated data on disk but issues its reads in one round; Starling spends about 60 bytes per point (2 GB for 33 million points) on PQ codes, a navigation graph and a vertex-to-block map, and gets more out of each read. BANG spends the most: 74 bytes per point on the GPU plus the whole graph and dataset, about 390 bytes per point, in host DRAM. Its throughput gains are real, but they answer a different question from DiskANN's, which was how to avoid paying for DRAM.

Build and updates. Only HNSW and FilteredVamana insert points natively; no paper among the seven evaluates deletions. DiskANN's five-day merged build and NSG's

k-NN-graph dependence make rebuilding expensive, and Starling's layout must be recomputed after every rebuild. Starling suggests handling updates at the database level by merging a small in-memory index into the static one [3], which keeps the static index static.

Comparability of the evidence. The headline numbers in Table II come from different datasets, recall definitions, hardware and baselines. DiskANN's 95% is 1-recall@1 on a 64 GB workstation; SPANN's comparison reran DiskANN on SPANN's 128 GB machine; Starling's comparison is within 2 GB/10 GB segments that its design targets and that cap SPANN's replication; BANG's speedups are over GPU baselines only. The authorship also matters. Harsha Vardhan Simhadri co-authors DiskANN, Filtered-DiskANN and BANG, and Ravishankar Krishnaswamy the first two of these. SPANN comes from a different Microsoft group. Starling (Zhejiang University and Zilliz) is the only evaluation of DiskANN from outside Microsoft, but it evaluates its own system, so it is independent rather than neutral.

Who wins where. On a single node with spare SSD capacity and tight latency, SPANN's one-round reads are the reported winner at matched memory. When disk space is constrained, as in a database segment, a graph index wins, and Starling's layout makes it substantially cheaper per query. For label-filtered queries, Filtered-DiskANN is the only design here with native support. When a GPU and a large host memory are available and throughput matters more than latency, BANG wins over other GPU methods. For data that fits in RAM, HNSW, NSG and in-memory Vamana remain the right tools. DiskANN itself is no longer the reported winner in any of these settings, but most of the later systems are built on its graph.

V. New Ideas and Justification

A. The stack view

The later papers each change one layer while taking the others as given. Starling re-lays out a graph it did not build; BANG searches a graph it did not build; Filtered-DiskANN changes the graph but not its layout. A simple cost model shows why the layers interact. If a query must expand ℓ vertices and each read makes u of them available, the query needs about ℓ/u reads. DiskANN has $u = 1$ by construction. Block search raises u only if the block-mates it gets for free are vertices the search actually needs, which depends on the layout. And the layout can only group vertices that the *pruning rule* has made neighbors. Starling's own numbers fit this model: on BIGANN the search path fell from 362 to 182 and ξ rose from 0.0625 to 0.344 [3].

B. Layout-aware pruning

Our initial proposal was to make RobustPrune aware of the block assignment, so that an edge to a vertex in the same block is accepted more easily (a larger α for such candidates in (1)). We call this rule *V1, favour co-located*. While implementing it we noticed a problem with the reasoning. Under block search, whenever p is expanded its whole block has already been read, so an edge from p to a block-mate does not save a read; it spends one of the R out-edge slots on something the search gets anyway. This suggests the opposite rule, *V2, block-mates as implicit edges*: when pruning p, merge all of p's block-mates into the sorted candidate list, let each one cover farther candidates exactly as in (1), but never store an edge to it. The freed slots go to candidates in other blocks. In Example 1, if a and p shared a block, V2 would drop the stored edge to a but still use a to decide that b is covered.

Both rules are run as one extra Vamana pass over a graph that has already been built and laid out with BNF, with the layout held fixed so that the graph is pruned for the blocks it will be read from. Neither adds memory or disk; the cost is one pass of construction.

C. Experiment

Setup. SIFT1M was not available in our environment, so we used synthetic data, which also lets us control the effective dimension: $n = 10^6$ points in 128 dimensions drawn from 100 Gaussian clusters, each with a random orientation and variances decaying as j^{-1} across directions (an effective dimension, by participation ratio, of about 18), and 1,000 held-out queries with exact ground truth. To check variance we repeated everything with three seeds at $n = 10^5$. We follow Starling's BIGANN configuration: $R = 31$, 256-byte records, 16 records per 4 KB block. Vamana uses $L = 75$ and $\alpha = 1.2$. BNF runs for Starling's default of 8 iterations, and block search uses $\sigma = 0.3$. We count block reads per query and simulate the disk: vectors stay in RAM and we use exact distances for routing instead of PQ, which isolates the effect of graph and layout. We compare: (A) DiskANN-style vertex search on the ID layout; (B) block search on the ID layout; (C) Starling-style block search on the BNF layout; (D) a control that gives C's graph a third, ordinary Vamana pass and keeps C's layout; (E) V1 with $\alpha\tau$, $\tau = 1.25$, for same-block candidates; and (F) V2. E and F also make one extra pass on C's layout, so D separates the effect of layout awareness from the effect of simply making another pass.

TABLE III
Block reads per query at fixed recall@10, $n = 10^6$ (mean of 1,000 queries). D, E and F all keep C's layout fixed (D is `D2` in our code).

Configuration	OR(G)	@0.90	@0.95	@0.98
A. DiskANN: vertex search, ID layout	0.000	71.2	134.4	308.2
B. Block search, ID layout	0.000	50.0	98.8	251.9
C. Starling: block search, BNF	0.093	28.4	44.7	82.7
D. C + one ordinary pass (control)	0.092	26.9	40.8	75.6
E. C + one V1 pass (initial rule)	0.097	26.9	40.8	75.6
F. C + one V2 pass	0.000	26.3	40.4	76.4

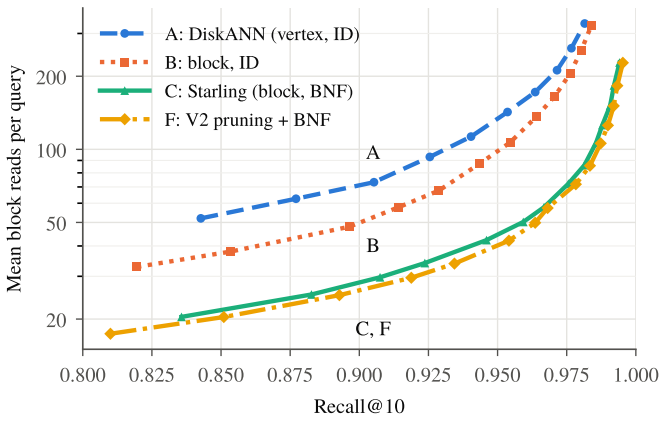


Fig. 2. Mean block reads per query against recall@10 at $n = 10^6$ (log scale). D and E overlap F and are omitted.

Results. Table III gives the results at $n = 10^6$ and Fig. 2 the full curves; the three runs at 10^5 show the same pattern.

Layout plus block search is the large effect. Starling-style C needs 2.5×, 3.0× and 3.7× fewer reads than DiskANN-style A at recall 0.90, 0.95 and 0.98 (3.0×, 3.7× and 4.4× on average at 10^5), and each block it reads holds about twice as many of the query's true 100 nearest neighbors (1.29 against 0.62 near recall 0.95). Block search on the unshuffled layout (B) already saves 18–30% of reads, because the extra vertices it evaluates act as extra entry points; it also performs 15 extra distance computations per read, which a read count does not see. Starling argues that without locality this extra computation does not pay for itself in time [3], and our measurement cannot test that.

The pruning rules add little beyond an extra pass. An ordinary third pass (D) cuts reads by 5–9% relative to C. V1, our initial rule, raises $\mathrm{OR}(G)$ from 0.092 to 0.097 and leaves reads unchanged relative to D, at both scales. This is not a coincidence: V1 altered only about 1% of edges, and 96% of the edges it added point to block-mates that block search reads anyway, so its curve coincides with D's. V2 removes every stored edge to a block-mate, so its $\mathrm{OR}(G)$ is zero by construction, and it lands within about 2% of D in both directions. At 10^5, where blocks hold twice as much locality, V2 needed 2–7% fewer reads than D in all three seeds and at every recall level, a small but consistent gain that disappears at 10^6. For scale, at 10^6 re-running BNF on the D and E graphs moved reads by up to 4% at recall 0.90–0.95 but by up to 14% at 0.98, so small differences at the high-recall end are noise.

Why. Two observations explain why the gain is small and shrinks with n. First, locality is scarce: at 10^6 BNF reaches $\mathrm{OR}(G) = 0.093$, so a vertex stores only about 1.4 of its 31 edges into its own block (0.093×15), leaving little for V2 to reclaim or V1 to add. $\mathrm{OR}(G)$ also halved from about 0.18 at 10^5, as Starling observed it falling with dataset size [3]. Second, $\mathrm{OR}(G)$ is the wrong target for co-design: V1 raises it and V2 sets it to zero, yet both read about the same number of blocks. It rewards stored edges that block search makes redundant. A better objective would score a layout by how often a block's vertices are actually expanded together during search, which is what saves reads. The usual caveats apply: synthetic data, exact rather than PQ routing, and reads rather than time; the code and data generator are in our submission, and replacing the generator with a data loader would repeat the run on SIFT1M or a BIGANN subset.

D. Secondary proposals

Two further ideas follow the same logic; we justify them analytically but did not test them. *Filter-aware layout.* Under a label filter of specificity s, a block laid out without regard to labels contains on average $15s$ block-mates the filtered search may use: 0.15 at $s = 1\%$. Starling's locality therefore does almost nothing for filtered queries, while grouping vertices by label inside BNF's neighborhoods could make most of the 16 records usable. The cost is a conflict between geometric and label locality that the layout would have to balance. *Block-level entry points.* Keeping one representative per block in memory costs $n/16$ entries, 6.25% of the points, comparable to Starling's 10% navigation sample, and would let the search start from a block rather than a vertex, as SPANN starts from a posting list.

VI. Future Issues and Implications

Updates. Every index reviewed here except HNSW and FilteredVamana is built once and treated as static, and none of the seven papers evaluates deletions, while production vector stores insert and delete continuously. Layout makes this harder: a block layout that is good for one graph decays as edges change, and Starling's answer, periodic rebuild and reshuffle, is expensive at DiskANN's build times.

Query semantics. Filtered-DiskANN handles one label per query, or a disjunction by merging searches. Real queries combine conjunctions of labels, numeric ranges and text constraints with vector similarity, and Starling shows that range search behaves very differently from top-k search on disk. Hybrid structured-plus-vector queries are an open problem for disk-resident graphs.

Hardware. The parameters these systems tune, α, R, beam width, block size and σ, are really properties of the device. Faster NVMe devices, larger blocks, GPU-direct storage and pooled memory change the relative price of a round trip, a read and a distance computation, so α and the layout should arguably be chosen per device rather than per dataset. BANG and Starling's future work both point toward GPUs; how to combine BANG's GPU arithmetic with a disk-resident graph rather than a DRAM-resident one remains open.

Evaluation. The billion-scale challenge at NeurIPS'21 gave the field shared datasets and baselines [11], yet, as Table II shows, the papers still report different recall definitions on different hardware against different baselines. Cost-normalized metrics such as SPANN's vectors-times-QPS per unit of memory [1], and reporting reads and distance computations alongside time, would make results comparable.

VII. Conclusion

In-memory graphs such as HNSW and NSG answer the question of how few distance computations a query needs. DiskANN changed the question to how few disk round trips it needs, and answered it with a pruning rule that keeps longer edges, a merge that builds a billion-point graph in 64 GB, and a layout that puts each vertex's vector next to its neighbors. SPANN showed that an inverted index with careful duplication can beat it at matched memory when disk is plentiful. The later papers each improve one layer: Starling the use of each block read, Filtered-DiskANN the queries a graph can answer, and BANG the hardware that walks it. We argued that these layers interact and tested the narrowest version of that claim. Our simulation supports the layout half of it, since block layout and block search cut reads about threefold, but only weakly supports the pruning half: the better pruning rule saved a few percent of reads beyond an ordinary extra construction pass, and nothing at a million points, while the locality score both rules were built around turned out to measure redundancy as much as usefulness. The next step is to score layouts and pruning rules by the reads a search actually saves, and to test them on real data with PQ routing and measured time.

References

[1] Q. Chen *et al.*, "SPANN: Highly-efficient billion-scale approximate nearest neighborhood search," in *Advances in Neural Information Processing Systems*, 2021.

[2] S. Gollapudi *et al.*, "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proceedings of the ACM Web Conference 2023 (WWW '23)*, 2023, pp. 3406–3416. doi: 10.1145/3543507.3583552.

[3] M. Wang *et al.*, "Starling: An I/O-efficient disk-resident graph index framework for high-dimensional vector similarity search on data segment," *Proceedings of the ACM on Management of Data*, vol. 2, no. 1, pp. 1–27, Feb. 2024, doi: 10.1145/3639269.

[4] P. Indyk and R. Motwani, "Approximate nearest neighbors: Towards removing the curse of dimensionality," in *Proceedings of the 30th Annual ACM Symposium on Theory of Computing (STOC)*, 1998, pp. 604–613. doi: 10.1145/276698.276876.

[5] S. Jayaram Subramanya, F. Devvrit, H. V. Simhadri, R. Krishnaswamy, and R. Kadekodi, "DiskANN: Fast accurate billion-point nearest neighbor search on a single node," in *Advances in Neural Information Processing Systems*, 2019.

[6] V. Karthik, S. Khan, S. Singh, H. V. Simhadri, and J. Vedurada, "BANG: Billion-scale approximate nearest neighbour search using a single GPU," *IEEE Transactions on Big Data*, vol. 11, no. 6, pp. 3142–3157, 2025, doi: 10.1109/TBDATA.2025.3581085.

[7] C. Fu, C. Xiang, C. Wang, and D. Cai, "Fast approximate nearest neighbor search with the navigating spreading-out graph," *Proceedings of the VLDB Endowment*, vol. 12, no. 5, pp. 461–474, 2019, doi: 10.14778/3303753.3303754.

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: 10.1109/TPAMI.2018.2889473.

[9] G. T. Toussaint, "The relative neighbourhood graph of a finite planar set," *Pattern Recognition*, vol. 12, no. 4, pp. 261–268, 1980, doi: 10.1016/0031-3203(80)90066-7.

[10] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 1, pp. 117–128, 2011, doi: 10.1109/TPAMI.2010.57.

[11] H. V. Simhadri *et al.*, "Results of the NeurIPS'21 challenge on billion-scale approximate nearest neighbor search," in *Proceedings of the NeurIPS 2021 Competitions and Demonstrations Track*, in Proceedings of Machine Learning Research, vol. 176. 2022, pp. 177–189.